DATA ANALYTICS | PORTFOLIO PROJECT

Olist E-Commerce Sales & Operations Analytics

End-to-end analysis of sales performance, order behaviour, product categories and delivery operations.

Prepared by
Vishal Bajpai

Technology stack
Python | PostgreSQL | SQL | Power BI | DAX

Dataset
Brazilian E-Commerce Public Dataset by Olist

Document version
1.2 | 05 November 2026

Portfolio project documentation based on a public dataset.

Executive Summary

This project analyses the Brazilian E-Commerce Public Dataset by Olist to evaluate commercial performance and fulfilment operations. The work follows an end-to-end analytics process: data exploration and preparation in Python, relational storage and SQL analysis in PostgreSQL, and interactive reporting in Power BI.

The latest dashboard reports R$13.59 million in item-price sales across 99.44 thousand orders. It also reports 96.48 thousand delivered orders, a 91.88% on-time delivery rate, a 0.63% cancellation rate, an average delivery time of 12.50 days and an average item price of R$120.65. These figures represent the latest dashboard state and remain subject to final source-level reconciliation.

Project at a glance

TOTAL SALES	TOTAL ORDERS	ON-TIME DELIVERY
R$13.59M	99.44K	91.88%
CANCELLATION RATE	**DELIVERED ORDERS**	**AVG. DELIVERY TIME**
0.63%	96.48K	12.50 days

The purpose is not limited to presenting metrics. It is to define measures consistently, understand source-data structure, validate relationships and filters, investigate unusual patterns, and translate results into practical business questions.

Report structure

Section	Coverage
01 | Business context	Business problem, objectives and scope
02 | Data foundation	Dataset, tables, grain and architecture
03 | Analytical workflow	Python, SQL, data modelling and DAX
04 | Dashboard and results	Dashboard design, KPI definitions and findings
05 | Recommendations	Commercial and operational opportunities
06 | Validation and next steps	Limitations, data checks and future enhancements

01 | Business Context

Business problem

E-commerce businesses require a reliable view of sales, order demand, product performance and fulfilment quality. Revenue alone does not explain whether orders are reaching customers on time, which product categories contribute most to sales, or whether changes in reported performance reflect genuine business activity or data-quality issues.

Project question: How can raw e-commerce data be transformed into a reliable, interactive view of commercial and operational performance that supports monitoring and decision-making?

Objectives

- Explore the source datasets and understand their structure, data types and relationships.
- Assess missing values, duplicate keys, status-specific nulls and date coverage.
- Analyse order volumes, item sales, category performance and delivery outcomes.
- Use PostgreSQL and SQL to query and validate relational data.
- Build a Power BI model with clear relationships, DAX measures and interactive filters.
- Communicate findings, assumptions, limitations and recommendations.

Scope

The current dashboard covers order status, item-price sales, monthly trends, product-category sales, on-time delivery, cancellation rate, average delivery time, customer geography and payment type. Profitability, marketing attribution, inventory optimisation and seller scorecards are outside the current scope because the required cost, campaign and inventory context has not been incorporated.

02 | Data Foundation

Dataset overview

The project uses the **Brazilian E-Commerce Public Dataset by Olist**, a public dataset containing linked information about orders, order items, products, customers, payments and other marketplace entities. The current reporting model focuses on the core tables required for sales and operations analysis.

Table	Analytical purpose
orders	Order identifier, status, purchase timestamp and delivery milestones.
order_items	Products associated with orders, including item price and freight.
products	Product identifiers and category attributes.
product_category_name_translation	English translations of product-category names.
customers	Customer identifiers and state/location attributes.
order_payments	Payment type and amount; an order may have multiple payment rows.
DateTable	Calendar dimension for date filtering and monthly reporting.

Data grain and modelling considerations

The **orders** table is generally one row per order, while **order_items** and **order_payments** may contain multiple rows per order. Order counts should use distinct order IDs, and joins must be designed to avoid multiplying item or payment records.

Source dataset

Kaggle: Brazilian E-Commerce Public Dataset by Olist
https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

Recommended repository structure

```
olist-ecommerce-sales-operations-analysis/
■■■ data/          # Dataset documentation; avoid raw data upload
■■■ notebooks/     # Python exploration and cleaning
■■■ sql/           # Database setup and analysis queries
■■■ powerbi/       # PBIX report and dashboard screenshots
■■■ reports/       # Findings and project documentation
■■■ README.md
■■■ requirements.txt
```

03 | End-to-End Analytical Workflow

Stage 1 — Data acquisition and setup

The source CSV files were obtained from Kaggle and organised for analysis. The workflow separates raw inputs from prepared outputs and assigns a clear role to each technology: Python for exploration and preparation, PostgreSQL for relational storage and SQL analysis, and Power BI for the semantic model and reporting.

Stage 2 — Python: loading, profiling and preparation

Pandas was used to load the CSV files, inspect dimensions and data types, profile missing values and prepare timestamp fields. The initial orders table contained **99,441 rows and 8 columns**. The duplicate order-ID check returned zero duplicate IDs in the inspected orders table.

- Converted order timestamp fields to datetime values for date-based analysis.
- Profiled missing values: order_approved_at (160), order_delivered_carrier_date (1,783) and order_delivered_customer_date (2,965) in the initial inspection.
- Reviewed missing timestamps by order status to distinguish expected lifecycle gaps from potential data issues.
- Calculated delivery duration and compared actual delivery dates with estimated dates for delivered orders.
- Merged order items with product attributes and category translations for category-level analysis.

A missing value is not automatically an error. An order that has not been delivered may legitimately have no customer-delivery timestamp. Null handling should reflect the order lifecycle rather than applying blanket deletion or replacing missing values with zero.

Stage 3 — Exploratory data analysis

Exploratory analysis examined order-status composition, monthly order activity, category sales contribution and delivery performance. The analysis focused on which statuses dominate, how demand changes over time, which categories generate the most item sales, and how reliably delivered orders meet their estimated delivery dates.

Stage 4 — PostgreSQL: relational storage and SQL analysis

The project database was created as **olist_analytics**. Relevant CSV datasets were imported and queried to validate record counts, status distribution, monthly trends and category-level sales. SQL views were explored to support monthly reporting.

```
-- Orders by status
SELECT order_status,
       COUNT(DISTINCT order_id) AS total_orders
FROM orders
GROUP BY order_status
ORDER BY total_orders DESC;

-- Total item-price sales
SELECT SUM(price) AS total_item_sales
FROM order_items;

-- Delivered orders
SELECT COUNT(DISTINCT order_id) AS delivered_orders
FROM orders
WHERE order_status = 'delivered';
```

A previously created monthly summary view ended earlier than the latest timestamp in the raw orders table. Monthly reporting should therefore be reconciled against base tables and a correctly related calendar dimension rather than relying on a potentially incomplete or disconnected summary view.

04 | Power BI Model & Measures

Data model

Power BI combines orders, order items, products, customers, payments and category translations with a dedicated DateTable. The preferred model uses one-to-many relationships and single-direction filtering wherever practical.

One-side table / field	Many-side table / field	Relationship
DateTable[Date]	orders[Order Date]	1 : * (active)
orders[order_id]	order_items[order_id]	1 : *
products[product_id]	order_items[product_id]	1 : *
customers[customer_id]	orders[customer_id]	1 : *
orders[order_id]	order_payments[order_id]	1 : *
category translation[name]	products[category]	1 : *

A date-only Order Date field is derived from the purchase timestamp and linked to DateTable[Date]. Monthly chart axes should use DateTable[Month Year], sorted by a numeric YearMonthSort field. Avoid relying on a disconnected pre-aggregated monthly view for interactive visuals.

Payment-type slicer behaviour

Because an order can have multiple payment records, payment-type filtering requires deliberate design. With single-direction relationships, a payment slicer may not automatically filter order and item measures. Validate filter propagation using a controlled approach, such as a tested TREATAS measure. Avoid enabling bidirectional filtering across the entire model without checking for ambiguous paths.

Core DAX measures

```
Total Sales =
SUM('public order_items'[price])

Total Orders =
DISTINCTCOUNT('public orders'[order_id])

Total Delivered Orders =
CALCULATE(
    DISTINCTCOUNT('public orders'[order_id]),
    'public orders'[order_status] = "delivered"
)

Cancellation Rate =
DIVIDE(
    CALCULATE(
        DISTINCTCOUNT('public orders'[order_id]),
        'public orders'[order_status] = "canceled"
    ),
    DISTINCTCOUNT('public orders'[order_id]),
    0
)

On-Time Delivery Rate =
VAR OnTimeOrders =
    CALCULATE(
        DISTINCTCOUNT('public orders'[order_id]),
```

```
        'public orders'[order_status] = "delivered",
        FILTER(
            'public orders',
            NOT ISBLANK('public orders'[order_delivered_customer_date]) &&
            'public orders'[order_delivered_customer_date]
                <= 'public orders'[order_estimated_delivery_date]
        )
    )
VAR DeliveredOrders =
    CALCULATE(
        DISTINCTCOUNT('public orders'[order_id]),
        'public orders'[order_status] = "delivered"
    )
RETURN DIVIDE(OnTimeOrders, DeliveredOrders, 0)

Average Delivery Time (Days) =
AVERAGEX(
    FILTER(
        'public orders',
        'public orders'[order_status] = "delivered" &&
        NOT ISBLANK('public orders'[order_delivered_customer_date])
    ),
    DATEDIFF(
        'public orders'[order_purchase_timestamp],
        'public orders'[order_delivered_customer_date],
        DAY
    )
)
```

Total Sales currently sums order_items[price]. It is an item-price sales measure, not profit and not necessarily total customer spend. Freight, discounts, refunds and costs should only be included if the business definition explicitly requires them.

05 | Power BI Dashboard

The following visual captures the latest dashboard version supplied for this report. It brings together executive KPIs, date and business-dimension slicers, monthly sales and order trends, category contribution and order-status composition.

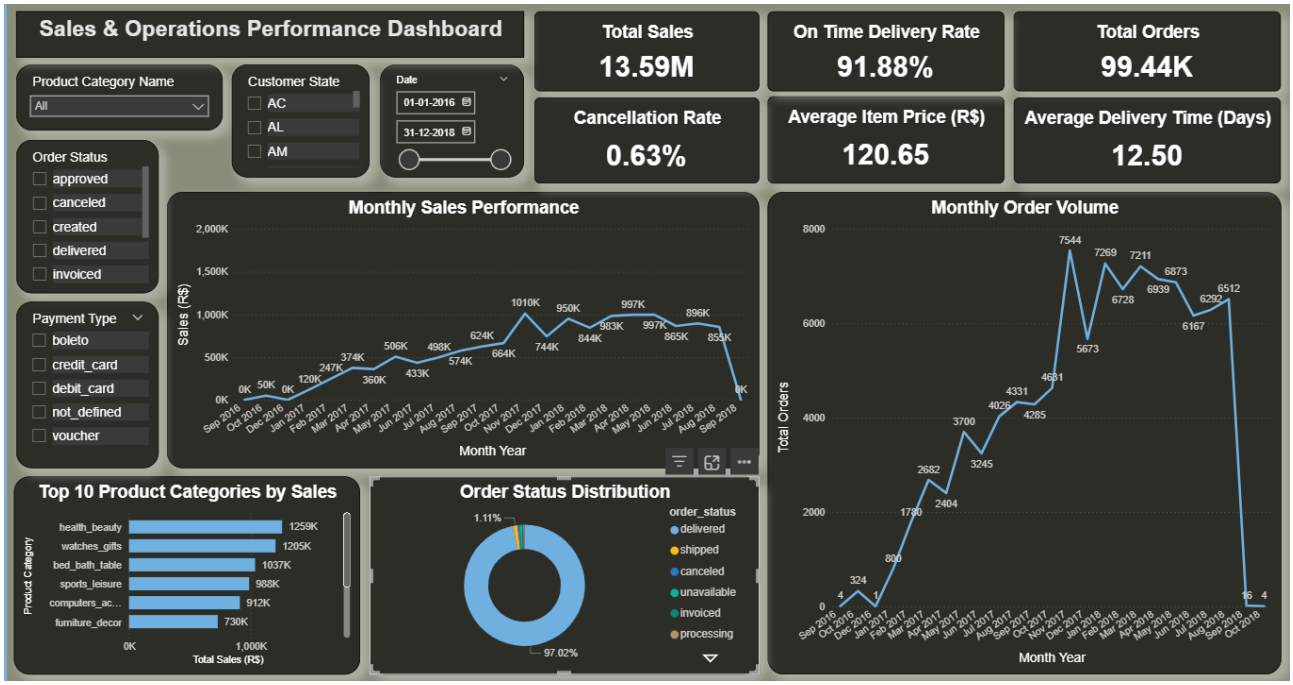


Figure 1. Sales & Operations Performance Dashboard — latest project version supplied by the author.

Dashboard components

Component	Purpose
KPI cards	Summarise sales, order volume, on-time delivery, cancellations, average item price and average delivery time.
Monthly Sales Performance	Tracks item-price sales over time.
Monthly Order Volume	Tracks distinct order volume over time.
Top 10 Product Categories	Ranks categories by item-price sales.
Order Status Distribution	Shows the composition of order lifecycle statuses.
Interactive slicers	Filter by category, customer state, date, order status and payment type.

The dashboard is designed to support both an executive overview and guided exploration. KPI cards provide a concise performance summary, while trend charts and slicers allow users to investigate changes by period, category, geography, order status and payment method.

06 | Results & Business Findings

Metric	Dashboard result	Definition
Total Sales	R$13.59M	Sum of item prices in the current model.
Total Orders	99.44K	Distinct order count.
Delivered Orders	96.48K	Orders with delivered status.
On-Time Delivery Rate	91.88%	On-time delivered orders / delivered orders.
Cancellation Rate	0.63%	Canceled orders / all orders in current filter context.
Average Delivery Time	12.50 days	Average purchase-to-delivery duration for delivered orders.
Average Item Price	R$120.65	Average item price under current measure and filter context.

Observed findings

- Health & Beauty is the leading category by sales in the current category analysis.
- Watches & Gifts and Bed, Bath & Table are among the other leading categories.
- The current on-time delivery measure reports 91.88% of delivered orders delivered on or before the estimated date.
- The 0.63% cancellation rate is consistent with approximately 625 canceled orders out of 99,441 total orders.
- Unavailable orders are a separate status and should not be silently combined with cancellations.
- Monthly sales and order-volume trends rise over much of the observed period, but the sharp end-of-period decline requires validation.

The cancellation rate uses canceled orders as the numerator and all orders as the denominator. If canceled and unavailable orders are combined into a broader operational exception metric, that metric should be named and reported separately.

07 | Recommendations & Validation

Business recommendations

- **Delivery operations:** Monitor on-time performance by month, state, seller and category. Investigate clusters of late deliveries rather than relying only on the overall rate.
- **Category planning:** Use category sales trends to guide merchandising and inventory discussions. Combine sales with margin and stock information before making profitability decisions.
- **Order exceptions:** Track canceled and unavailable orders separately and investigate their causes, timing and geographic concentration.
- **Customer geography:** Compare sales, order volume and delivery performance by state to identify service gaps and potential growth opportunities.
- **Payment analysis:** Compare payment methods while accounting for multiple payment rows per order and validating slicer behaviour.
- **Data quality:** Establish repeatable checks for row counts, unique keys, date coverage, null patterns and reconciliation totals across Python, SQL and Power BI.

Priority validation plan

Validation area	Required action
Date coverage	Compare earliest and latest purchase timestamps across raw data, SQL and Power BI.
Monthly totals	Aggregate orders and item sales by month from base tables and reconcile chart values.
Date relationship	Confirm DateTable[Date] actively filters orders[Order Date].
Slicer behaviour	Test each slicer independently and in combination; confirm relevant KPIs and charts respond.
Order grain	Use DISTINCTCOUNT(order_id) for order counts; avoid summing duplicated order-level data.
Sales definition	Label Total Sales as item-price sales if freight, discounts and refunds are excluded.
Payment grain	Account for multiple payment rows per order and validate payment-type filtering.
KPI denominator	Document whether percentages use all orders or delivered orders.

Open investigation — September 2018 sales decline

A sharp decline appears towards the end of the monthly sales and order-volume charts. The raw orders table was previously observed to contain a latest purchase timestamp of 17 October 2018, while an earlier monthly summary view ended in August 2018. This difference makes it important to validate source coverage, reporting periods, calendar relationships, visual filters and monthly aggregation before interpreting the decline as a business event.

Project limitations

- The sales measure sums item prices and does not represent net profit or necessarily total customer spend.
- Product costs, margin, marketing investment and inventory data are not included in the current model.
- The end-of-period decline is an open validation item, not a confirmed business explanation.

- Dashboard figures are development outputs and should be reconciled against final source-level control totals before external use.
- Average delivery time may vary depending on date granularity, null handling and filter context.

SQL Analysis & Query Results

Supplement to the Olist E-Commerce Sales & Operations Analytics report | Prepared by Vishal Bajpai

Purpose. This appendix adds the PostgreSQL queries shared during project development, the observed query outputs and concise analytical interpretations. The original 12-page report is retained unchanged.

Output note. Monthly result tables reproduce only the rows visible in the supplied screenshots; they are not represented as complete query outputs.

Query 01 — Total Order Count

Objective: Establish the baseline number of records in the orders table.

```
SELECT COUNT(*) AS total_orders
FROM orders;
```

total_orders
99,441

Interpretation: The query returned 99,441 order records. This is the baseline order volume for the project.

Query 02 — Order Status Distribution

Objective: Count orders by status and calculate each status's percentage of all orders.

```
SELECT
order_status,
COUNT(*) AS total_orders,
ROUND(
COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (),
2
) AS percentage
FROM orders
GROUP BY order_status
ORDER BY total_orders DESC;
```

order_status	total_orders	percentage (%)
delivered	96,478	97.02
shipped	1,107	1.11
canceled	625	0.63
unavailable	609	0.61
invoiced	314	0.32
processing	301	0.30
created	5	0.01
approved	2	0.00
TOTAL	99,441	100.00

Finding: Delivered orders account for 97.02%. Canceled and unavailable orders are separate lifecycle statuses and should be analysed independently.

Query 03 — Monthly Order Trend

Objective: Aggregate order volume by purchase month.

```
SELECT
DATE_TRUNC('month', order_purchase_timestamp)::DATE
AS order_month,
COUNT(*) AS total_orders
FROM orders
GROUP BY
DATE_TRUNC('month', order_purchase_timestamp)::DATE
ORDER BY order_month;
```

order_month	total_orders
2016-09-01	4
2016-10-01	324
2016-12-01	1
2017-01-01	800
2017-02-01	1,780
2017-03-01	2,682
2017-04-01	2,404
2017-05-01	3,700
2017-06-01	3,245
2017-07-01	4,026
2017-08-01	4,331
2017-09-01	4,285
2017-10-01	4,631
2017-11-01	7,544
2017-12-01	5,673
2018-01-01	7,269
2018-02-01	6,728
2018-03-01	7,211
2018-04-01	6,939
2018-05-01	6,873
2018-06-01	6,167

Finding: Monthly order volume increased during 2017. November 2017 is the highest visible month at 7,544 orders. Screenshot rows are visible through June 2018 only.

SQL concepts: DATE_TRUNC groups timestamps into months; GROUP BY aggregates each month; ORDER BY sorts chronologically.

Query 04 — Average Delivery Time

Objective: Calculate elapsed time from purchase to customer delivery for delivered orders with valid timestamps.

```
SELECT
COUNT(*) AS delivered_orders,
ROUND(
AVG(
EXTRACT(
EPOCH FROM (
order_delivered_customer_date
- order_purchase_timestamp
)
) / 86400
)::NUMERIC,
2
) AS avg_delivery_days
FROM orders
WHERE order_status = 'delivered'
AND order_delivered_customer_date IS NOT NULL
AND order_purchase_timestamp IS NOT NULL;
```

delivered_orders	avg_delivery_days
96,470	12.56

Interpretation: Average purchase-to-delivery time is 12.56 days. EPOCH converts the interval to seconds; division by 86,400 converts seconds to days.

Validation: The original dashboard showed 12.50 days. Reconcile filters and date logic between PostgreSQL and Power BI.

Query 05 — Average Delivery Delay

Objective: Compare actual customer delivery date with estimated delivery date.

```
SELECT
COUNT(*) AS delivered_orders,
ROUND(
AVG(
EXTRACT(
EPOCH FROM (
order_delivered_customer_date
- order_estimated_delivery_date
)
) / 86400
)::NUMERIC,
2
) AS avg_delivery_delay_days
FROM orders
WHERE order_status = 'delivered'
AND order_delivered_customer_date IS NOT NULL
AND order_estimated_delivery_date IS NOT NULL;
```

delivered_orders	avg_delivery_delay_days
96,470	-11.18

Interpretation: The negative result means that actual delivery occurred, on average, 11.18 days before the estimated date. Negative = early; positive = late.

Query 06 — On-Time Delivery Rate

Objective: Calculate the share of eligible delivered orders delivered on or before the estimated date.

```
SELECT
COUNT(*) AS delivered_orders,
COUNT(*) FILTER (
WHERE order_delivered_customer_date
<= order_estimated_delivery_date
) AS on_time_orders,
ROUND(
100.0 * COUNT(*) FILTER (
WHERE order_delivered_customer_date
<= order_estimated_delivery_date
) / COUNT(*),
2
) AS on_time_delivery_rate
FROM orders
WHERE order_status = 'delivered'
AND order_delivered_customer_date IS NOT NULL
AND order_estimated_delivery_date IS NOT NULL;
```

delivered_orders	on_time_orders	on_time_delivery_rate (%)
96,470	88,644	91.89

Interpretation: 88,644 of 96,470 eligible delivered orders arrived on time, giving a rate of 91.89%. FILTER applies a condition to an aggregate.

Validation: The original Power BI report showed 91.88%. Reconcile eligible records, filter context and rounding.

Query 07 — Top 10 Product Categories by Sales

Objective: Rank categories by item-price sales and show item counts and average item price.

```
SELECT
COALESCE(
p.product_category_name,
'unknown'
) AS product_category,
COUNT(*) AS total_items,
ROUND(SUM(oi.price), 2) AS total_sales,
ROUND(AVG(oi.price), 2) AS average_item_price
FROM order_items oi
LEFT JOIN products p
ON oi.product_id = p.product_id
GROUP BY
COALESCE(
p.product_category_name,
'unknown'
)
ORDER BY total_sales DESC
LIMIT 10;
```

product_category	total_items	total_sales (R$)	average_item_price (R$)
beleza_saude	9,670	1,258,681.34	130.16
relogios_presentes	5,991	1,205,005.68	201.14
cama_mesa_banho	11,115	1,036,988.68	93.30
esporte_lazer	8,641	988,048.97	114.34
informatica_acessorios	7,827	911,954.32	116.51
moveis_decoracao	8,334	729,762.49	87.56
cool_stuff	3,796	635,290.85	167.36
utilidades_domesticas	6,964	632,248.66	90.79
automotivo	4,235	592,720.11	139.96
ferramentas_jardim	4,347	485,256.46	111.63

Findings: beleza_saude leads the displayed categories at R$1,258,681.34. cama_mesa_banho has the highest item count (11,115); relogios_presentes has the highest average item price (R$201.14).

Metric definition: SUM(oi.price) is item-price sales, not profit or total customer spend. Freight, discounts, refunds and costs are excluded.

Query 08 — Monthly Sales & Delivered Order Performance

Objective: Compare monthly delivered-order volume, item-price sales and average item price.

```
SELECT
DATE_TRUNC(
'month',
o.order_purchase_timestamp
)::DATE AS order_month,
COUNT(DISTINCT o.order_id) AS total_orders,
ROUND(SUM(oi.price), 2) AS total_sales,
ROUND(AVG(oi.price), 2) AS average_item_price
FROM orders o
JOIN order_items oi
ON o.order_id = oi.order_id
WHERE o.order_status = 'delivered'
GROUP BY
DATE_TRUNC(
'month',
o.order_purchase_timestamp
)::DATE
ORDER BY order_month;
```

order_month	total_orders	total_sales (R$)	average_item_price (R$)
2016-09-01	1	134.97	44.99
2016-10-01	265	40,325.11	128.83
2016-12-01	1	10.90	10.90
2017-01-01	750	111,798.36	122.45
2017-02-01	1,653	234,223.40	126.06
2017-03-01	2,546	359,198.85	123.99
2017-04-01	2,303	340,669.68	132.61
2017-05-01	3,546	489,338.25	122.21
2017-06-01	3,135	421,923.37	120.93
2017-07-01	3,872	481,604.52	109.06
2017-08-01	4,193	554,699.70	115.63
2017-09-01	4,150	607,399.67	128.22
2017-10-01	4,478	648,247.65	124.33
2017-11-01	7,289	987,765.37	116.55
2017-12-01	5,513	726,033.19	117.35
2018-01-01	7,069	924,645.00	115.05
2018-02-01	6,555	826,437.13	109.93
2018-03-01	7,003	953,356.25	118.92
2018-04-01	6,798	973,534.09	124.38
2018-05-01	6,749	977,544.69	125.17
2018-06-01	6,099	856,077.86	122.12

Findings: November 2017 is the highest visible month by delivered orders (7,289) and recorded R$987,765.37 in item-price sales. May 2018 recorded R$977,544.69, followed by a decline in June to R$856,077.86.

Grain note: COUNT(DISTINCT order_id) counts unique orders; SUM(price) and AVG(price) operate at item-row grain. Average item price is not average order value. This query includes delivered orders only.

Consolidated SQL Notes & Validation

SQL practice	Why it matters
COUNT(DISTINCT order_id)	Avoids counting an order more than once when it has multiple item rows.
DATE_TRUNC('month', timestamp)	Creates consistent monthly reporting periods.
LEFT JOIN	Retains item rows even if product information is missing.
COALESCE	Labels missing category names as 'unknown'.
FILTER (WHERE ...)	Performs conditional aggregation, such as counting on-time deliveries.
EXTRACT(EPOCH FROM interval)	Converts timestamp differences into seconds for duration calculations.
Explicit WHERE conditions	Defines the eligible population used in each KPI.

Reconciliation items before publication

Area	Observed values / action
Total orders	COUNT(*) returned 99,441.
Delivered orders	Status distribution returned 96,478; delivery-duration queries returned 96,470 after requiring non-null date fields.
Average delivery time	SQL returned 12.56 days; original dashboard showed 12.50 days. Reconcile date logic and filter context.
On-time delivery	SQL returned 91.89%; original dashboard showed 91.88%. Reconcile eligible records and calculation context.
Monthly results	Only screenshot-visible rows are reproduced here; export full query results before treating as exhaustive.
Sales definition	Sales queries sum order_items.price; do not describe this as profit or net revenue.

Next investigation: Validate the September 2018 sales decline using source date coverage, monthly base-table totals, order-status composition, calendar relationships and dashboard filters. Do not treat the decline as a confirmed business event until these checks are complete.